Лабораторная работа 1 по МВМ
Решение системы линейных уравнений методом QR-разложения
Выполнил: студент группы А-14-23 Кузнецов Давид
Вариант 9 (нечётный)

1. Постановка задачи

Необходимо решить систему линейных алгебраических уравнений

$$Ax = b \tag{1}$$

с помощью QR-разложения, получаемого вращениями Гивенса.

Матрица A порядка N имеет ленточную структуру:

$$a_{ii} = \min\{N, 10\}, \qquad a_{i,i\pm j} = \frac{1}{j}, \quad j = 1, \ldots, K. \tag{2}$$

Элементы, индексы которых выходят за границы матрицы, не задаются.

Расчёты проводятся для двух случаев:

- заполненная матрица, для которой $K = N$;
- ленточная матрица, для которой $K = \lfloor 0.1N \rfloor$.

Точное решение выбирается в виде

$$x_i^* = \begin{cases} 7, & i \text{ нечётно}, \\ 0, & i \text{ чётно}, \end{cases} \qquad i = 1, \ldots, N. \tag{3}$$

После этого правая часть вычисляется по формуле

$$b = Ax^*. \tag{4}$$

Абсолютная погрешность полученного решения определяется в евклидовой норме:

$$\varepsilon = \|x - x^*\|_2. \tag{5}$$

Так как вариант нечётный, используется гауссовский порядок обхода элементов: сначала выполняются вращения в плоскостях

$$(1,2), (1,3), \ldots, (1,N),$$

затем аналогично обрабатываются следующие столбцы.

Матрица Q явно не формируется. Вращения сразу применяются как к строкам матрицы A, так и к элементам правой части b.

2. Метод вращений Гивенса

Для зануления элемента

$$\beta = a_{ji},$$

расположенного под диагональю, вместе с опорным элементом

$$\alpha = a_{ii}$$

вычисляются следующие величины:

$$r = \sqrt{\alpha^2 + \beta^2}, \qquad c = \cos\varphi = \frac{\alpha}{r}, \qquad s = \sin\varphi = \frac{\beta}{r}. \tag{6}$$

Матрица вращения имеет вид

$$G = \begin{pmatrix} c & s \\ -s & c \end{pmatrix}. \tag{7}$$

К двум выбранным строкам матрицы применяются преобразования

$$R_{i,:}^{new} = cR_{i,:}^{old} + sR_{j,:}^{old}, \tag{8}$$

$$R_{j,:}^{new} = -sR_{i,:}^{old} + cR_{j,:}^{old}. \tag{9}$$

Аналогичные преобразования применяются к правой части:

$$y_i^{new} = cy_i^{old} + sy_j^{old}, \tag{10}$$

$$y_j^{new} = -sy_i^{old} + cy_j^{old}. \tag{11}$$

После последовательного зануления всех элементов под главной диагональю получается верхняя треугольная система

$$Rx = y. \tag{12}$$

Полученная система решается методом обратной подстановки.

При работе с ленточной матрицей сохраняются только элементы исходной ленты и дополнительной верхней части, возникающей при вращениях.

Для хранения используется массив размера

$$N \times (3K + 1). \tag{13}$$

Нулевые элементы за пределами рабочей области не хранятся и не обрабатываются.

Раздел A. Подробный расчёт при $N = 4$

Для заполненной матрицы при

$$N = 4, \qquad K = N$$

получаем исходную матрицу

$$A = \begin{pmatrix} 4 & 1 & 0.5 & 0.333333 \\ 1 & 4 & 1 & 0.5 \\ 0.5 & 1 & 4 & 1 \\ 0.333333 & 0.5 & 1 & 4 \end{pmatrix}. \tag{14}$$

Точное решение имеет вид

$$x^* = \begin{pmatrix} 7 \\ 0 \\ 7 \\ 0 \end{pmatrix}. \tag{15}$$

Правая часть равна

$$b = Ax^* = \begin{pmatrix} 31.5 \\ 14 \\ 31.5 \\ 9.33333 \end{pmatrix}. \tag{16}$$

Таблица 1. Параметры вращений Гивенса

Шаг	Столбец	Плоскость	$\cos\varphi$	$\sin\varphi$
1	1	$(1,2)$	0.970143	0.242536
2	1	$(1,3)$	0.992727	0.120386
3	1	$(1,4)$	0.996795	0.0800000
4	2	$(2,3)$	0.978915	0.204269
5	2	$(2,4)$	0.995970	0.0896901
6	3	$(3,4)$	0.979231	0.202749

Параметры вращений.

Промежуточные матрицы. После обработки первого столбца получаем

$$R^{(1)} = \begin{pmatrix} 4.16667 & 2.08000 & 1.28000 & 0.880000 \\ 0 & 3.63803 & 0.848875 & 0.404226 \\ 0 & 0.759144 & 3.88332 & 0.939198 \\ 0 & 0.334673 & 0.900486 & 3.94224 \end{pmatrix}. \tag{17}$$

После обработки второго столбца:

$$R^{(2)} = \begin{pmatrix} 4.16667 & 2.08000 & 1.28000 & 0.880000 \\ 0 & 3.73143 & 1.69844 & 0.938763 \\ 0 & 0 & 3.62804 & 0.836824 \\ 0 & 0 & 0.751181 & 3.87365 \end{pmatrix}. \tag{18}$$

После обработки третьего столбца получается итоговая верхняя треугольная матрица

$$R = R^{(3)} = \begin{pmatrix} 4.16667 & 2.08000 & 1.28000 & 0.880000 \\ 0 & 3.73143 & 1.69844 & 0.938763 \\ 0 & 0 & 3.70499 & 1.60482 \\ 0 & 0 & 0 & 3.62353 \end{pmatrix}. \tag{19}$$

Преобразованная правая часть равна

$$y = Q^T b = \begin{pmatrix} 38.1267 \\ 11.8890 \\ 25.9349 \\ -8.88178 \cdot 10^{-16} \end{pmatrix}. \tag{20}$$

Следовательно, необходимо решить треугольную систему

(21)
$$\begin{pmatrix} 4.16667 & 2.08000 & 1.28000 & 0.880000 \\ 0 & 3.73143 & 1.69844 & 0.938763 \\ 0 & 0 & 3.70499 & 1.60482 \\ 0 & 0 & 0 & 3.62353 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 38.1267 \\ 11.8890 \\ 25.9349 \\ -8.88178 \cdot 10^{-16} \end{pmatrix}.$$

В результате обратной подстановки получено решение

(22)
$$x = \begin{pmatrix} 7 \\ 0 \\ 7 \\ -2.45114 \cdot 10^{-16} \end{pmatrix}.$$

Абсолютная погрешность решения равна

(23)
$$\|x - x^*\|_2 = 1.27977 \cdot 10^{-15}.$$

Невязка равна

(24)
$$\|Ax - b\|_2 = 5.02430 \cdot 10^{-15}.$$

Полученное решение совпадает с заданным точным решением с точностью машинного округления.

Раздел Б. Скорость решения и проверка трудоёмкости

Время решения измерялось с помощью функции

`perf_counter`.

Расчёты на используемом компьютере выполнены до значения

$$N = 2000.$$

Значение $N = 10000$ для заполненной матрицы не запускалось из-за ограничений оперативной памяти и времени. Один массив размера

$$10000 \times 10000$$

с элементами типа `float64` занимает приблизительно 763 Мбайт.

В программе предусмотрен флаг `run_n10000`. Расчёт можно включить на компьютере с достаточным объёмом оперативной памяти.

Для соседних размеров N_1 и N_2 показатель роста времени вычисляется по формуле

$$p_{time} = \frac{\ln(T_2/T_1)}{\ln(N_2/N_1)}. \tag{25}$$

Показатель роста условного объёма работы определяется по формуле

$$p_{work} = \frac{\ln(W_2/W_1)}{\ln(N_2/N_1)}. \tag{26}$$

Здесь T — измеренное время, а W — условный объём обработки:

$$W(N,K) = \sum_{i=0}^{N-2} \min(K, N-i-1)\,\min(N-i, 2K+1). \tag{27}$$

Таблица 2. Результаты для заполненной матрицы

N	K	Время, с	Погрешность	p_{time}	p_{work}
4	4	0.000161875	$1.27977 \cdot 10^{-15}$	–	–
10	10	0.000793833	$2.66250 \cdot 10^{-15}$	1.73531	3.05947
50	50	0.0362562	$3.08264 \cdot 10^{-14}$	2.37443	3.00600
100	100	0.0926828	$6.99574 \cdot 10^{-14}$	1.35407	3.00043
500	500	1.33005	$3.41539 \cdot 10^{-13}$	1.65511	3.00006
1000	1000	5.29508	$7.07716 \cdot 10^{-13}$	1.99317	3.00000
2000	2000	29.6090	$1.32828 \cdot 10^{-12}$	2.48331	3.00000

Заполненная матрица, $K = N$**.** На каждом из порядка N^2 вращений обрабатывается до N элементов. Поэтому трудоёмкость алгоритма имеет порядок

$$O(N^3). \tag{28}$$

Значения p_{work} быстро приближаются к 3, что подтверждает теоретическую оценку.

Таблица 3. Результаты для ленточной матрицы

N	K	Время, с	Погрешность	p_{time}	p_{work}
4	0	0.000027375	0	–	–
10	1	0.000106833	$3.38685 \cdot 10^{-15}$	1.48603	–
50	5	0.00205017	$1.21474 \cdot 10^{-14}$	1.83568	2.82189
100	10	0.00776954	$2.66521 \cdot 10^{-14}$	1.92209	2.93654
500	50	0.194906	$1.09548 \cdot 10^{-13}$	2.00213	2.97707
1000	100	0.853472	$3.99145 \cdot 10^{-13}$	2.13056	2.99318
2000	200	3.55212	$3.82022 \cdot 10^{-13}$	2.05726	2.99658

Ленточная матрица, $K = \lfloor 0.1N \rfloor$. Для ленточной матрицы на каждый столбец приходится не более K вращений. Каждое вращение обрабатывает $O(K)$ элементов.

Поэтому общая трудоёмкость равна

$$O(NK^2). \tag{29}$$

Поскольку в эксперименте

$$K = \lfloor 0.1N \rfloor,$$

получаем

$$O(NK^2) = O(N^3). \tag{30}$$

Значения p_{work} стремятся к 3 и подтверждают эту оценку.

Показатель p_{time} оказался меньше 3. Это не опровергает теоретическую оценку, поскольку операции над срезами выполняются в оптимизированных процедурах NumPy. Кроме того, время одного запуска зависит от накладных расходов, работы кэша и оперативной памяти.

Выводы

В работе реализовано решение системы линейных уравнений

$$Ax = b$$

с помощью QR-разложения вращениями Гивенса в гауссовском порядке обхода элементов.

Матрица Q явно не формировалась. Вращения одновременно применялись к матрице и правой части системы.

Для заполненной матрицы использовалось обычное хранение. Для ленточной матрицы применялось компактное хранение размера

$$N \times (3K + 1).$$

Благодаря этому элементы за пределами рабочей ленты не хранятся и не обрабатываются.

При $N = 4$ найденное решение совпало с точным решением с погрешностью порядка

$$10^{-15}.$$

Для систем до $N = 2000$ погрешности остаются малыми и имеют порядок от 10^{-15} до 10^{-12}.

Расчёт показателя p_{work} подтвердил кубическую асимптотику

$$O(N^3)$$

как для заполненной матрицы, так и для ленточной матрицы при

$$K = 0.1N.$$

Код

```
import csv
import numpy as np
from time import perf_counter


# 1. Формированиеточногорешения
def make_exact_solution(n):
   x = np.zeros(n, dtype=np.float64)
   x[::2] = 7.0
   return x


# 2. Фактическаяполушириналенты
def effective_bandwidth(n, k):
   if n <= 0:
      raise ValueError("Размер матрицыn долженбытьположительным")

   if k < 0:
      raise ValueError("Полуширина ленты k неможетбытьотрицательной")

   return min(k, n - 1)


# 3. Формированиеполнойматрицы

def make_dense_matrix(n, k):
   k = effective_bandwidth(n, k)
   matrix_a = np.zeros((n, n), dtype=np.float64)
   np.fill_diagonal(matrix_a, min(n, 10))

   for j in range(1, k + 1):
      side_element = 1.0 / j
      indices = np.arange(n - j)
      matrix_a[indices, indices + j] = side_element
      matrix_a[indices + j, indices] = side_element

   return matrix_a



# 4. Формированиеправойчасти

def make_right_side(n, k, x):
   k = effective_bandwidth(n, k)
   right_b = min(n, 10) * x.copy()

   for j in range(1, k + 1):
      side_element = 1.0 / j
      right_b[:-j] += side_element * x[j:]
      right_b[j:] += side_element * x[:-j]

   return right_b


# 5. QRразложение- полнойматрицы

def givens_qr_dense_gaussian(matrix_a, right_b, k, save_history=False):
   matrix_r = matrix_a.copy()
   y = right_b.copy()
   n = matrix_r.shape[0]
   k = effective_bandwidth(n, k)

   rotations = []
   matrices = []
```

```
65
66      for column in range(n - 1):
67          last_row = min(n, column + k + 1)
68          last_column = min(n, column + 2 * k + 1)
69
70          for row in range(column + 1, last_row):
71              alpha = matrix_r[column, column]
72              beta = matrix_r[row, column]
73
74              if beta == 0.0:
75                  continue
76
77              radius = np.hypot(alpha, beta)
78              cos_phi = alpha / radius
79              sin_phi = beta / radius
80
81              support_row = matrix_r[column, column:last_column].copy()
82              bottom_row = matrix_r[row, column:last_column].copy()
83
84              matrix_r[column, column:last_column] = cos_phi * support_row + sin_phi *
                    bottom_row
85              matrix_r[row, column:last_column] = -sin_phi * support_row + cos_phi *
                    bottom_row
86              matrix_r[row, column] = 0.0
87
88              old_y_support = y[column]
89              old_y_bottom = y[row]
90
91              y[column] = cos_phi * old_y_support + sin_phi * old_y_bottom
92              y[row] = -sin_phi * old_y_support + cos_phi * old_y_bottom
93
94              if save_history:
95                  rotations.append({"column": column, "row_support": column, "row_bottom":
                        row, "cos_phi": cos_phi, "sin_phi": sin_phi})
96
97          if save_history:
98              matrices.append(matrix_r.copy())
99
100     return matrix_r, y, rotations, matrices
101
102
103 # 6. Обратнаяподстановкадляобычнойматрицы
104 def back_substitution_dense(matrix_r, y, k):
105     n = len(y)
106     k = effective_bandwidth(n, k)
107     x = np.zeros(n, dtype=np.float64)
108
109     for i in range(n - 1, -1, -1):
110         last_column = min(n, i + 2 * k + 1)
111         known_part = np.dot(matrix_r[i, i + 1:last_column], x[i + 1:last_column])
112
113         if abs(matrix_r[i, i]) < np.finfo(np.float64).eps:
114             raise ArithmeticError(f"Нулевой диагональныйэлементR[{i}, {i}]")
115
116         x[i] = (y[i] - known_part) / matrix_r[i, i]
117
118     return x
119
120
121 # 7. Компактноехранениеленточнойматрицы
122
123 def make_compact_band_matrix(n, k):
124     k = effective_bandwidth(n, k)
125     compact_matrix = np.zeros((n, 3 * k + 1), dtype=np.float64)
126     compact_matrix[:, k] = min(n, 10)
127
```

```
   for j in range(1, k + 1):
      side_element = 1.0 / j
      compact_matrix[:n - j, k + j] = side_element
      compact_matrix[j:, k - j] = side_element

   return compact_matrix


# 8. Переводкомпактнойматрицывобычнуютолько( длявывода)

def compact_to_dense(compact_matrix, n, k):
   k = effective_bandwidth(n, k)
   dense_matrix = np.zeros((n, n), dtype=np.float64)

   for i in range(n):
      first_column = max(0, i - k)
      last_column = min(n, i + 2 * k + 1)

      for j in range(first_column, last_column):
         compact_column = k + j - i
         dense_matrix[i, j] = compact_matrix[i, compact_column]

   return dense_matrix


# 9. QRразложение- ленточнойматрицы

def givens_qr_band_gaussian(compact_matrix, right_b, k, save_history=False):
   matrix_r = compact_matrix.copy()
   y = right_b.copy()
   n = matrix_r.shape[0]
   k = effective_bandwidth(n, k)

   rotations = []
   matrices = []

   for column in range(n - 1):
      last_row = min(n, column + k + 1)
      last_column = min(n, column + 2 * k + 1)
      row_length = last_column - column

      for row in range(column + 1, last_row):
         support_start = k
         bottom_start = k + column - row

         alpha = matrix_r[column, support_start]
         beta = matrix_r[row, bottom_start]

         if beta == 0.0:
            continue

         radius = np.hypot(alpha, beta)
         cos_phi = alpha / radius
         sin_phi = beta / radius

         support_row = matrix_r[column, support_start:support_start + row_length].
             copy()
         bottom_row = matrix_r[row, bottom_start:bottom_start + row_length].copy()

         matrix_r[column, support_start:support_start + row_length] = cos_phi *
             support_row + sin_phi * bottom_row
         matrix_r[row, bottom_start:bottom_start + row_length] = -sin_phi *
             support_row + cos_phi * bottom_row
         matrix_r[row, bottom_start] = 0.0

         old_y_support = y[column]
```

```
            old_y_bottom = y[row]

            y[column] = cos_phi * old_y_support + sin_phi * old_y_bottom
            y[row] = -sin_phi * old_y_support + cos_phi * old_y_bottom

            if save_history:
                rotations.append({"column": column, "row_support": column, "row_bottom":
                    row, "cos_phi": cos_phi, "sin_phi": sin_phi})

        if save_history:
            matrices.append(compact_to_dense(matrix_r, n, k))

    return matrix_r, y, rotations, matrices



# 10. Обратнаяподстановкадляленточнойматрицы

def back_substitution_band(compact_r, y, k):
    n = len(y)
    k = effective_bandwidth(n, k)
    x = np.zeros(n, dtype=np.float64)

    for i in range(n - 1, -1, -1):
        last_column = min(n, i + 2 * k + 1)
        number_of_upper_elements = last_column - i - 1
        known_part = np.dot(compact_r[i, k + 1:k + 1 + number_of_upper_elements], x[i
            + 1:last_column])
        diagonal_element = compact_r[i, k]

        if abs(diagonal_element) < np.finfo(np.float64).eps:
            raise ArithmeticError(f"Нулевой диагональныйэлементR[{i}, {i}]")

        x[i] = (y[i] - known_part) / diagonal_element

    return x


# 11. Форматированиерезультатов

def format_array(array):
    """Выводит элементысшестьюзначащимицифрами."""
    return np.array2string(np.asarray(array), formatter={"float_kind": lambda value:
        f"{value:.6g}"}, max_line_width=150)


# 12. Подсчётусловногообъёмаработы

def count_work(n, k):
    k = effective_bandwidth(n, k)
    work = 0

    for column in range(n - 1):
        number_of_rotations = min(k, n - column - 1)
        number_of_columns = min(n - column, 2 * k + 1)
        work += number_of_rotations * number_of_columns

    return work



# 13. Подробныйвывод

def print_detailed_result(result):
    """Выводит подробнуюинформациюопроцессерешения."""
    n = result["n"]
```

```
254
255     print("=" * 70)
256     print(f"ПОДРОБНЫЙ РАСЧЁТ: {result['type'].upper()} МАТРИЦА, N = {n}, K = {result
            ['k']}")
257     print("=" * 70)
258
259     print("\Исходная\n матрицаA:")
260     print(format_array(result["A"]))
261
262     print("\Точное\n решениеx*:")
263     print(format_array(result["x_exact"]))
264
265     print("\Правая\n частьb = A*x*:")
266     print(format_array(result["b"]))
267
268     print("\Параметры\n вращенийГивенса:")
269
270     if len(result["rotations"]) == 0:
271         print("Вращения невыполнялись.")
272
273     for step, rotation in enumerate(result["rotations"], start=1):
274         print(f"Шаг {step}: столбец{rotation['column'] + 1}, плоскость({rotation['
                row_support'] + 1}, {rotation['row_bottom'] + 1}); cos(phi) = {rotation['
                cos_phi']:.6g}; sin(phi) = {rotation['sin_phi']:.6g}")
275
276     print("\Промежуточные\n матрицы:")
277
278     for column, matrix in enumerate(result["matrices"], start=1):
279         print(f"\После\n обработки{columnго}- столбца:")
280         print(format_array(matrix))
281
282     print("\Итоговая\n верхняятреугольнаяматрицаR:")
283     print(format_array(result["R"]))
284
285     print("\Преобразованная\n праваячастьy = Q^T*b:")
286     print(format_array(result["y"]))
287
288     print("\Полученная\n треугольнаясистемаR*x = y:")
289
290     for i in range(n):
291         coefficients = " ".join(f"{result['R'][i, j]:.6g}" for j in range(n))
292         print(f"[{coefficients}] x = {result['y'][i]:.6g}")
293
294     print("\Точное\n решение:")
295     print(format_array(result["x_exact"]))
296
297     print("\Найденное\n решение:")
298     print(format_array(result["x_calculated"]))
299
300     print(f"\Абсолютная\n погрешность||x - x*||_2 = {result['error']:.6g}")
301     print(f"Невязка ||A*x - b||_2 = {result['residual']:.6g}")
302     print(f"Время решения= {result['time']:.6g} с")
303
304
305 # 14. Общаяфункциярешенияоднойзадачи
306
307 def solve_problem(n, matrix_type="full", detailed=False):
308     """
309     matrix_type:
310         "full" –заполненнаяматрица;
311         "band" –ленточнаяматрица
312
313     detailed:
314         True –вывестиподробнуюинформацию;
315         False –вернутьтолькоитоговыерезультаты
316     """
```

```python
    if matrix_type not in ("full", "band"):
        raise ValueError('matrix_type долженбыть"full" или"band"')

    if matrix_type == "full":
        k = n
        matrix_name = "Заполненная"
    else:
        k = n // 10
        matrix_name = "Ленточная"

    x_exact = make_exact_solution(n)
    right_b = make_right_side(n, k, x_exact)

    if matrix_type == "full":
        matrix_a = make_dense_matrix(n, k)

        start = perf_counter()
        matrix_r, y, rotations, matrices = givens_qr_dense_gaussian(matrix_a, right_b,
             k, save_history=detailed)
        x_calculated = back_substitution_dense(matrix_r, y, k)
        elapsed = perf_counter() - start

        matrix_r_for_print = matrix_r

    else:
        compact_matrix = make_compact_band_matrix(n, k)
        matrix_a = make_dense_matrix(n, k) if detailed else None

        start = perf_counter()
        matrix_r, y, rotations, matrices = givens_qr_band_gaussian(compact_matrix,
            right_b, k, save_history=detailed)
        x_calculated = back_substitution_band(matrix_r, y, k)
        elapsed = perf_counter() - start

        matrix_r_for_print = compact_to_dense(matrix_r, n, k) if detailed else None

    absolute_error = np.linalg.norm(x_calculated - x_exact, ord=2)
    calculated_b = make_right_side(n, k, x_calculated)
    residual = np.linalg.norm(calculated_b - right_b, ord=2)

    result = {
        "type": matrix_name,
        "matrix_type": matrix_type,
        "n": n,
        "k": k,
        "time": elapsed,
        "error": absolute_error,
        "residual": residual,
        "work": count_work(n, k)
    }

    if detailed:
        result.update({
            "A": matrix_a,
            "b": right_b,
            "R": matrix_r_for_print,
            "y": y,
            "x_exact": x_exact,
            "x_calculated": x_calculated,
            "rotations": rotations,
            "matrices": matrices
        })

        print_detailed_result(result)

    return result
```

```python


# 15. РасчётыдляразличныхN

def run_series(sizes, matrix_type):
    """Решаетсистемудлякаждого
       N ивычисляет:

    p времени= ln(T2/T1) / ln(N2/N1);
    p работы= ln(W2/W1) / ln(N2/N1)
    """
    results = []
    previous_result = None

    if matrix_type == "full":
        title = "Заполненная матрица, K = N"
    else:
        title = "Ленточная матрица, K = floor(0.1*N)"

    print("\n" + title)
    print(f"{'N':>8} {'K':>8} Время{', c':>14} Погрешность{'':>16} {'p времени':>14}
        {'p работы':>14}")

    for n in sizes:
        result = solve_problem(n=n, matrix_type=matrix_type, detailed=False)

        if previous_result is None:
            result["p_time"] = np.nan
            result["p_work"] = np.nan
        else:
            logarithm_n = np.log(result["n"] / previous_result["n"])
            result["p_time"] = np.log(result["time"] / previous_result["time"]) /
                logarithm_n

            if previous_result["work"] > 0 and result["work"] > 0:
                result["p_work"] = np.log(result["work"] / previous_result["work"]) /
                    logarithm_n
            else:
                result["p_work"] = np.nan

        p_time_text = "–" if np.isnan(result["p_time"]) else f"{result['p_time']:.6g}"
        p_work_text = "–" if np.isnan(result["p_work"]) else f"{result['p_work']:.6g}"

        print(f"{result['n']:>8d} {result['k']:>8d} {result['time']:>14.6g} {result['
            error']:>16.6g} {p_time_text:>14} {p_work_text:>14}")

        results.append(result)
        previous_result = result

    return results


# 16. Сохранениерезультатов

def save_results_to_csv(filename, results):
    """Сохраняет результатывCSVфайл–."""
    with open(filename, "w", newline="", encoding="utf-8") as csv_file:
        writer = csv.writer(csv_file, delimiter=";")
        writer.writerow(["Тип матрицы", "N", "K", "Время, с", "Абсолютная погрешность"
            , "Невязка", "p времени", "p работы"])

        for result in results:
            p_time = "" if np.isnan(result["p_time"]) else f"{result['p_time']:.6g}"
            p_work = "" if np.isnan(result["p_work"]) else f"{result['p_work']:.6g}"
```

```
            writer.writerow([result["type"], result["n"], result["k"], f"{result['time
                ']:.6g}", f"{result['error']:.6g}", f"{result['residual']:.6g}", p_time
                , p_work])

    print(f"\Результатыn сохраненывфайл: {filename}")


# 17. Главнаячастьпрограммы

def main():
    # РазделА: подробныйрасчётдлязаполненнойматрицы
    solve_problem(n=4, matrix_type="full", detailed=True)

    #Для подробногорасчёталенточнойматрицысK > 0:

    #solve_problem(n=20, matrix_type="band", detailed=True)

    #При N = 20 получаемK = 2.

    sizes = [4, 10, 50, 100, 500, 1000, 2000]

    run_n10000 = False

    if run_n10000:
        sizes.append(10_000)

    print("\n" + "=" * 90)
    print("РАЗДЕЛ Б. ИЗМЕРЕНИЕВРЕМЕНИИПРОВЕРКАТРУДОЁМКОСТИ")
    print("=" * 90)

    full_results = run_series(sizes, matrix_type="full")
    band_results = run_series(sizes, matrix_type="band")

    all_results = full_results + band_results
    save_results_to_csv("benchmark_results.csv", all_results)


if __name__ == "__main__":
    main()
```